

OMU Mini Project

Final Report

Project Title: APU Campus Wide Drinks and Snack Ordering Kiosk System

Group Members	Dubernet Mathieu, Cavaignac Romain, Haegeman Victor
Subject	Object Modelling with UML
Submission	UML Report and Java Skeletal Implementation

Prepared for the OMU mini project assignment by Dubernet Mathieu, Cavaignac Romain, and Haegeman Victor.

Table of Contents

1. Introduction and Project Plan
2. Analysis and UML Design
3. Design Pattern Refinement
4. Implementation
5. Scenario Walkthrough
6. Compliance Assessment
7. Critical Appraisal
8. Conclusion and Future Enhancements
9. References

This report follows the same printable report organization as the provided CineRate report. UML diagrams are rendered from the PlantUML sources stored in `diagrams/plantuml`, with SVG images stored in `diagrams/rendered`.

1. Introduction and Project Plan

Asia Pacific University requires a campus-wide drinks and snack ordering kiosk system for students and staff. The system should reduce queues around the snack booth by allowing users to browse products, customize drinks or snacks, place an order, pay by AP card, and send the confirmed order to the preparation area.

The initial deployment includes five kiosks, with a possible expansion to ten kiosks. Because multiple kiosks access the same menu and order system, the design must support centralized product updates, clear separation of responsibilities, and enough concurrency awareness to avoid inconsistent product price or availability data.

1.1 Project Objectives

- Model the kiosk ordering scenario using UML use case, class, and sequence diagrams.
- Design a maintainable object-oriented system for drink and snack ordering.
- Apply two to three design patterns in a refined class design.
- Implement the selected patterns in a Java skeletal prototype.
- Evaluate the chosen patterns and the suitability of the solution.

1.2 Scope and End Users

The primary end users are students and staff who order from kiosks. Snack booth admin staff manage product prices and availability. Preparation staff receive order notifications, while the AP card service identifies users and deducts balances.

1.3 Major Functions

- Identify the user and display the current AP card balance.
- Browse available drinks and snacks.
- Customize items such as coffee size, sugar, milk, ice, toppings, or sandwich ingredients.
- Place orders and deduct payment from the AP card balance.
- Notify the snack booth preparation area.
- Log kiosk transactions for traceability.
- Update product prices and availability centrally.
- Synchronize product updates across all kiosks.

2. Analysis and UML Design

2.1 Use Case Model

The use case diagram identifies the customer, snack booth admin staff, preparation staff, and AP card service. It shows the main kiosk actions and supporting operations required by the assignment brief.

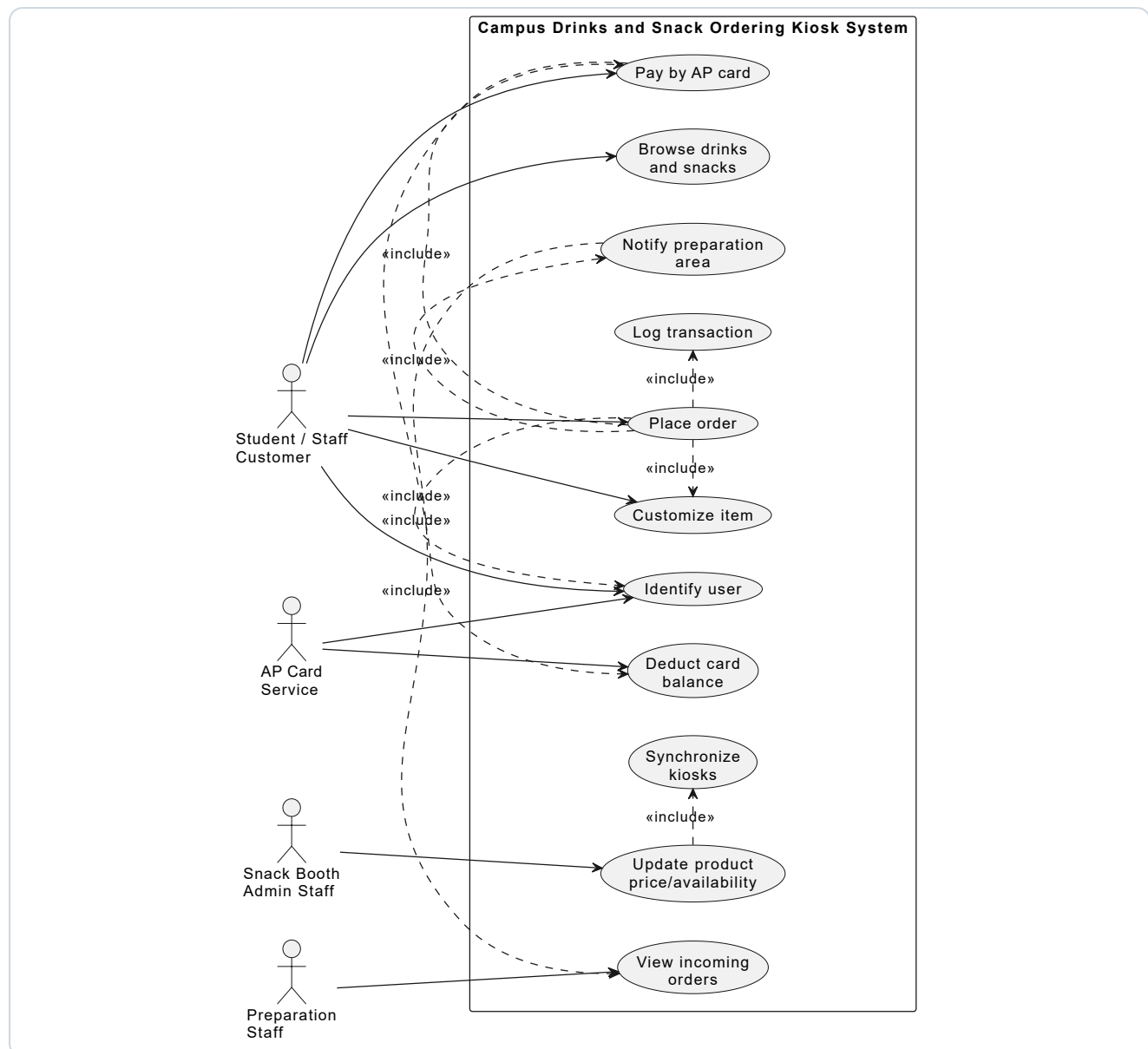


Figure 1. High-level use case diagram for the kiosk ordering system.

2.2 Main Use Case Descriptions

Use Case	Description	Main Actor
Identify user	The kiosk reads the AP card, obtains the user's identity, and displays the current balance before ordering.	Student / Staff Customer
Browse drinks and snacks	The user views currently available menu products synchronized from the central catalog.	Student / Staff Customer
Customize item	The user selects supported options such as size, sugar, ice, toppings, or ingredients.	Student / Staff Customer

Place order	The system creates the order, processes payment, logs the transaction, and notifies preparation staff.	Student / Staff Customer
Pay by AP card	The kiosk sends the order total to the payment service and confirms whether the user's AP card can cover the purchase.	Student / Staff Customer
Deduct card balance	The AP card service deducts the confirmed order amount and returns the updated balance.	AP Card Service
Notify preparation area	After payment succeeds, the order details are sent to the snack booth preparation area.	Kiosk System
Log transaction	The kiosk records the payment and order transaction for traceability and later auditing.	Kiosk System
Update product price/availability	Admin staff centrally change product data and trigger kiosk synchronization.	Snack Booth Admin Staff
Synchronize kiosks	Each kiosk receives the latest central catalog state so menus remain consistent across campus.	Central Catalog
View incoming orders	Preparation staff view confirmed orders with product, customization, user, and payment information.	Preparation Staff

2.3 Initial Class Diagram

The initial class diagram models the system before introducing design patterns. Domain classes represent users, AP cards, products, orders, order items, customizations, and transactions. Service classes coordinate kiosk, payment, catalog, logging, and notification responsibilities.

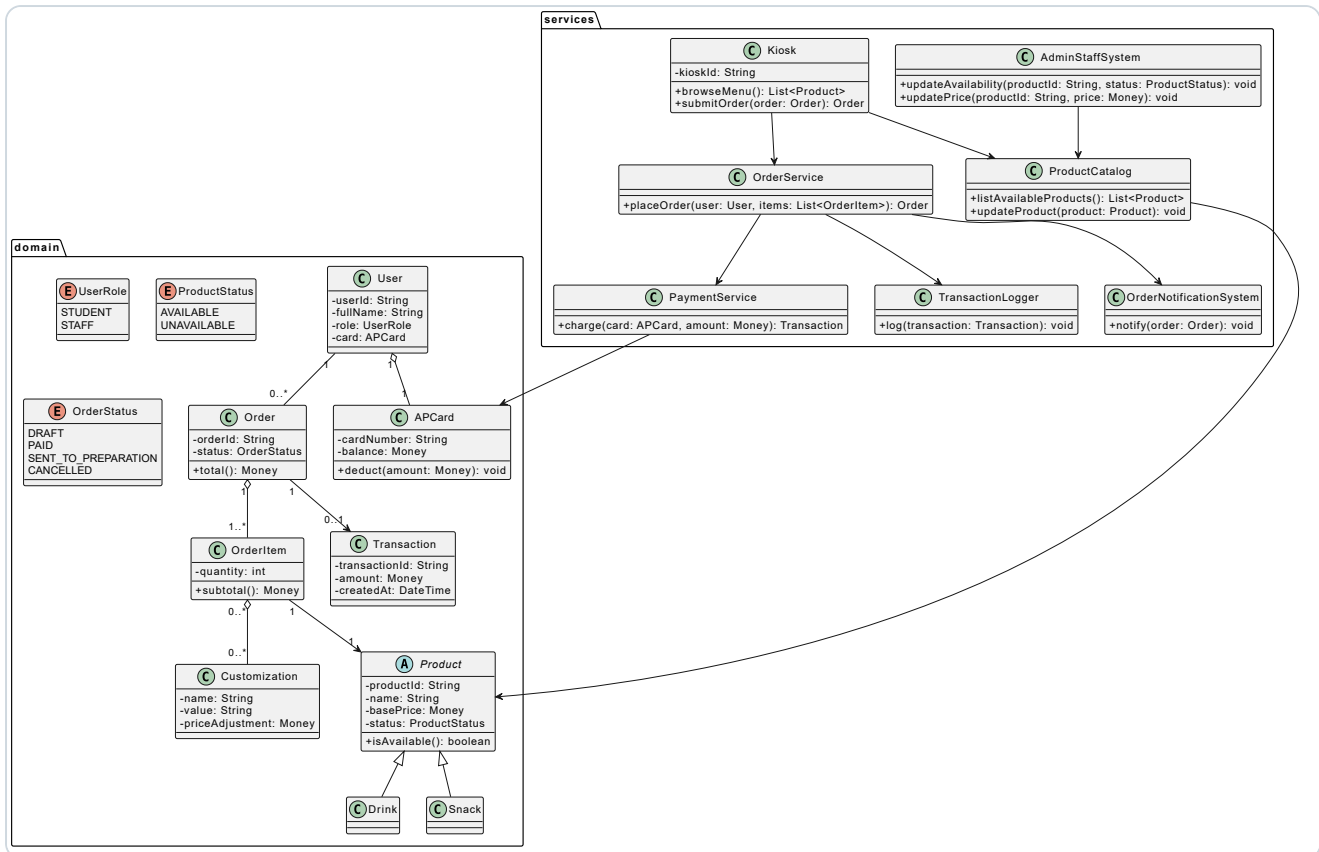


Figure 2. Initial class diagram without design patterns.

2.4 Sequence Diagrams

Three sequence diagrams describe core runtime behavior. The first covers browsing and customizing an item, the second covers order placement and payment, and the third covers central product updates from the admin system.

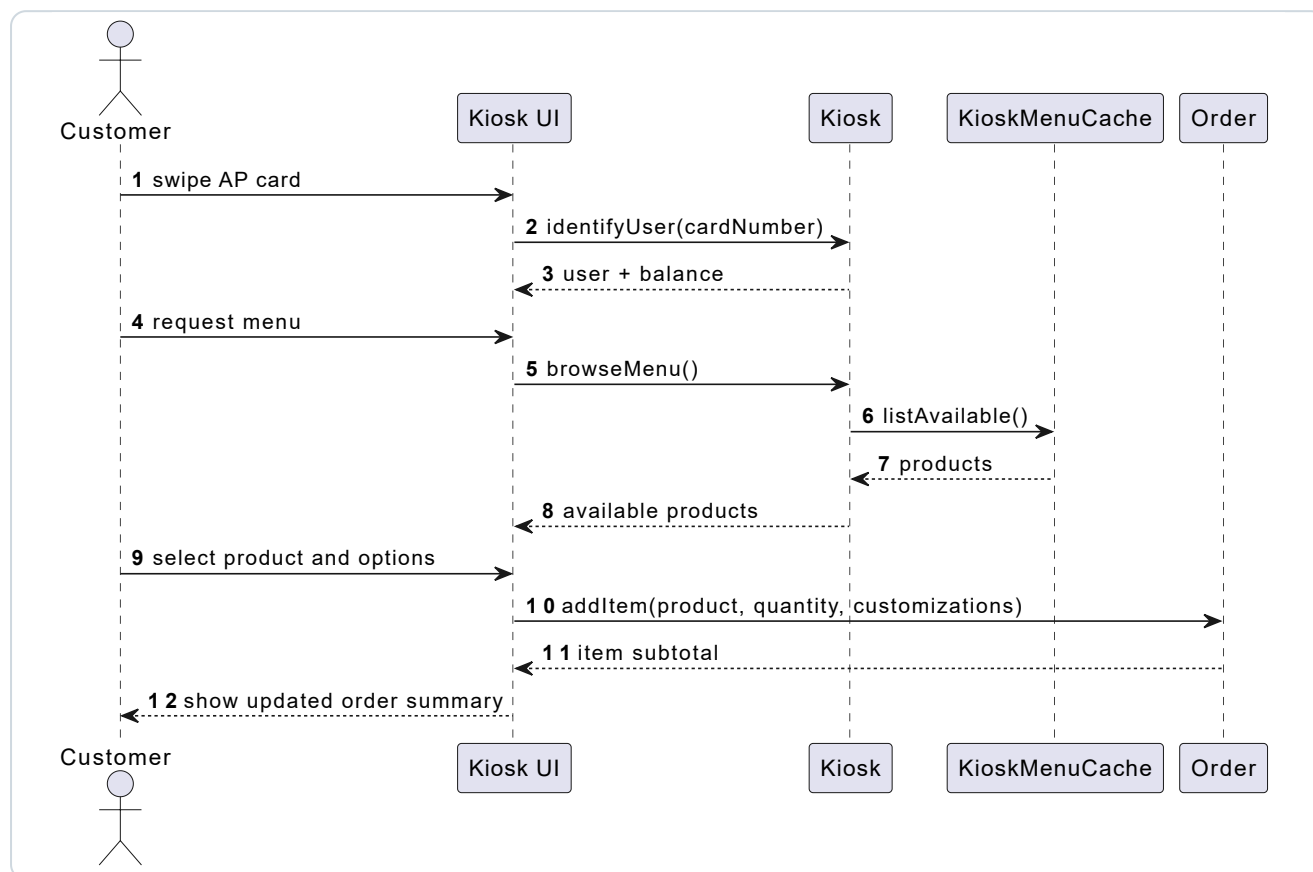


Figure 3. Sequence diagram for browsing and customizing an item.

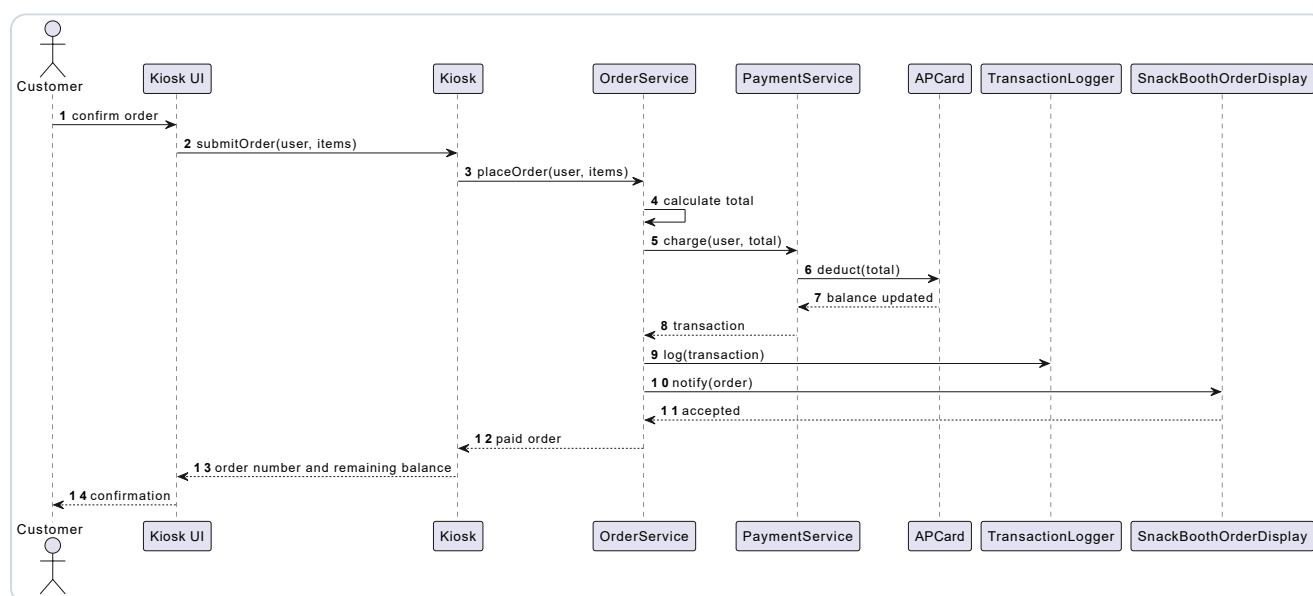


Figure 4. Sequence diagram for placing and paying for an order.

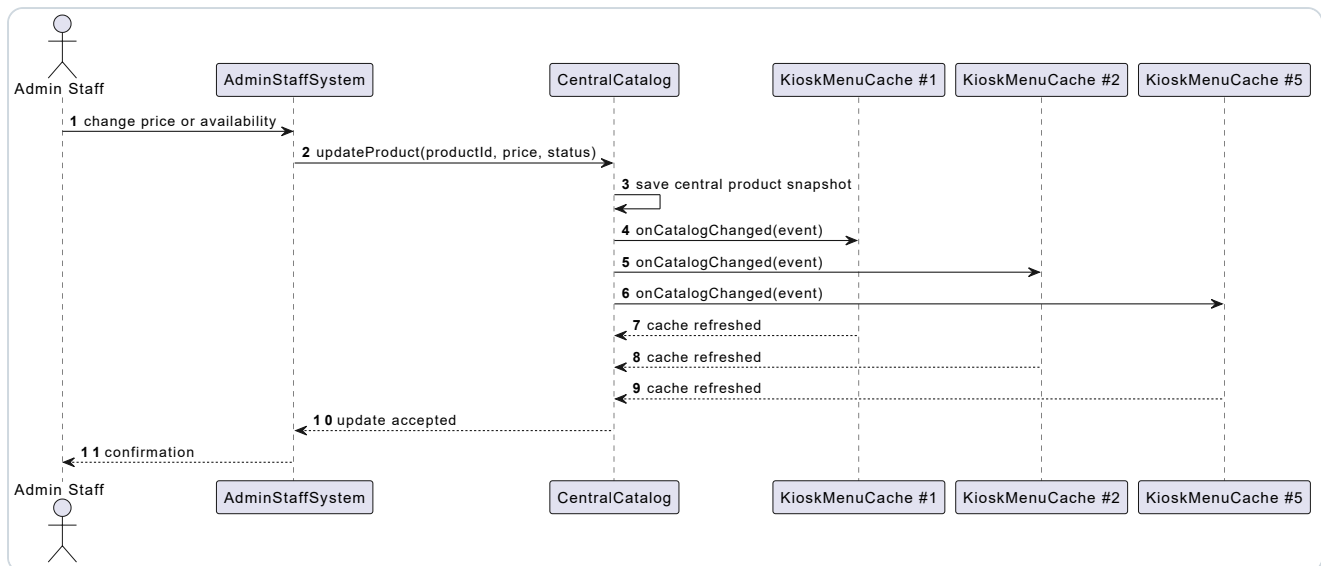


Figure 5. Sequence diagram for updating product information centrally.

3. Design Pattern Refinement

The refined solution introduces three design patterns that directly support the assignment goals of maintainability, extensibility, and central synchronization.

3.1 Selected Patterns

Pattern	Role in the Solution	Reason for Selection
Singleton	CentralCatalog acts as the single source of truth for product data.	Prevents each kiosk from maintaining conflicting product price or availability data.
Observer	KioskMenuCache observes catalog changes through CatalogObserver.	Allows all kiosks to react to central updates without hard-coding kiosk references.
Factory Method	DrinkFactory and SnackFactory create product subtypes.	Makes it easier to add new product families or product creation rules later.

3.2 Refined Class Diagram with Patterns

The refined class diagram shows how the design patterns are integrated into the object model. The pattern packages separate product creation, catalog observation, and central catalog access from the main business domain.

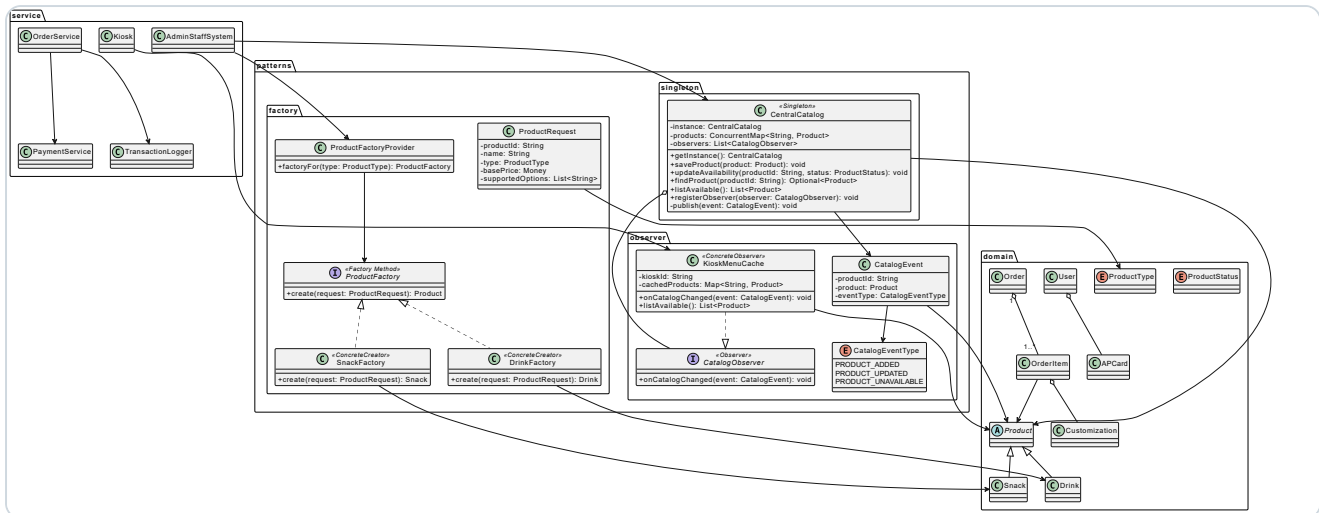


Figure 6. Refined class diagram with Singleton, Observer, and Factory Method.

3.3 Pattern Interaction Diagrams

The pattern sequence diagrams illustrate how the catalog update, factory creation, and singleton access flows operate at runtime.

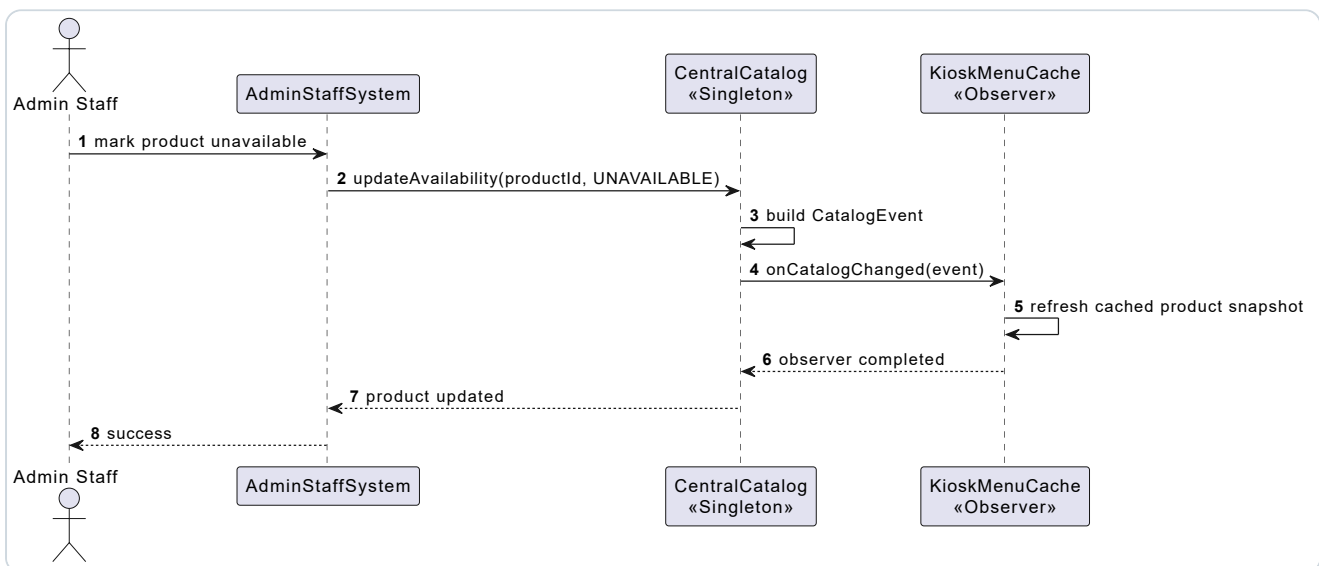


Figure 7. Observer sequence for catalog synchronization.

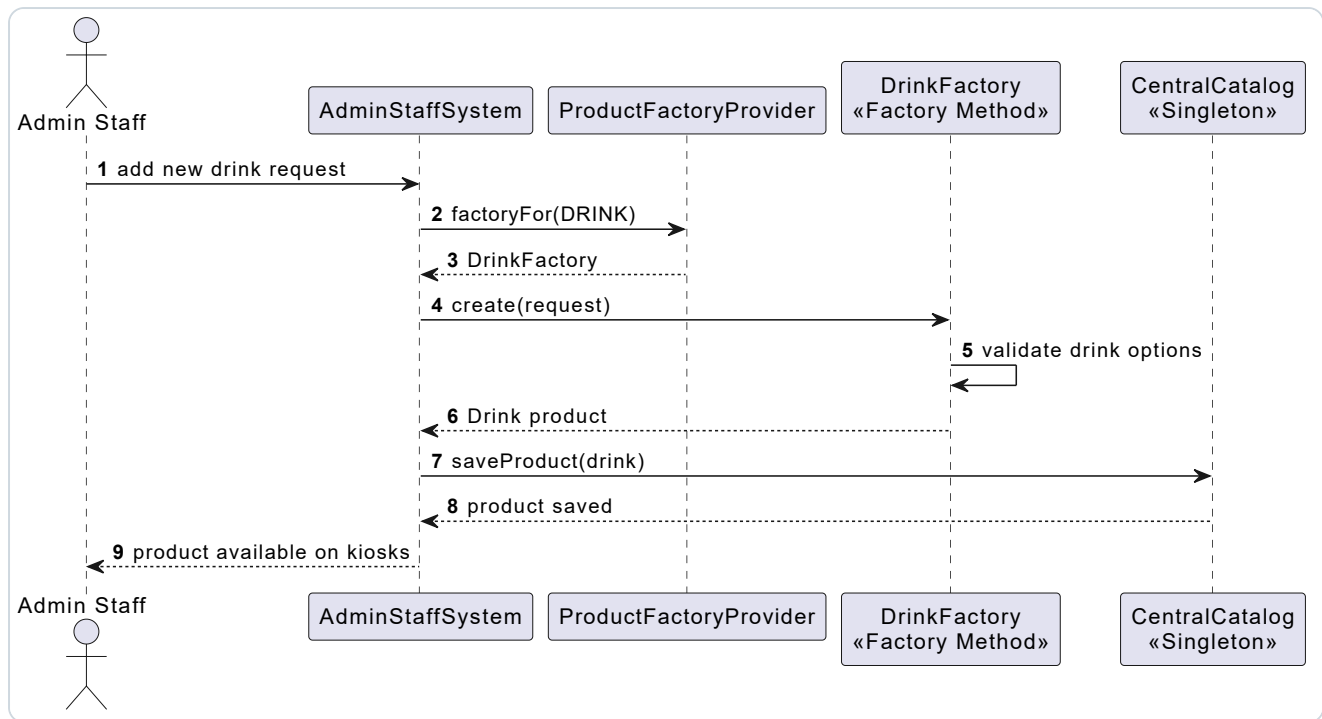


Figure 8. Factory Method sequence for product creation.

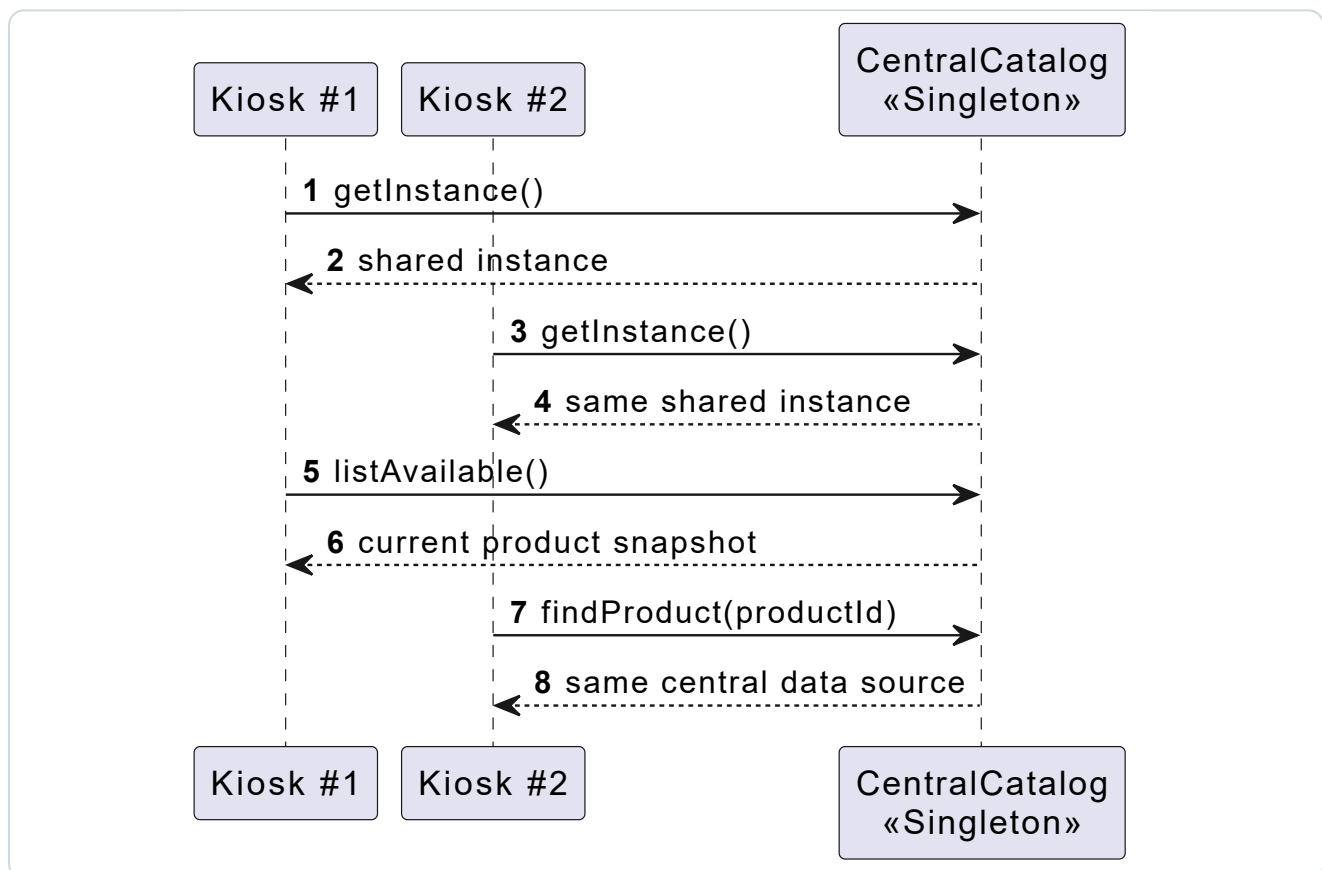


Figure 9. Singleton sequence for shared catalog access.

4. Implementation

4.1 Technical Structure

The Java implementation is skeletal and focuses on demonstrating the selected design patterns. It has no external runtime dependency and is organized into domain, service, and pattern packages.

```
src/main/java/omu
|-- domain
|   |-- APCard.java
|   |-- Product.java
|   |-- Drink.java
|   |-- Snack.java
|   |-- Order.java
|   |-- OrderItem.java
|   |-- Customization.java
|   |-- Transaction.java
|-- patterns
|   |-- factory
|   |-- observer
|   |-- singleton
|-- service
|   |-- AdminStaffSystem.java
|   |-- Kiosk.java
|   |-- OrderService.java
|   |-- PaymentService.java
|   |-- TransactionLogger.java
|   |-- SnackBoothOrderDisplay.java
|-- Main.java
```

4.2 Implemented Pattern Evidence

Code Area	Evidence
patterns/singleton/CentralCatalog.java	Provides a shared catalog instance and thread-aware product update methods.
patterns/observer	Defines catalog events and observers so kiosk menu caches can update automatically.
patterns/factory	Separates drink and snack creation through factory classes and product requests.
service/OrderService.java	Coordinates order creation, AP card payment, transaction logging, and preparation notification.

4.3 Verification

A simple scenario check is provided in `src/test/java/omu/PatternScenarioTest.java`. The local environment used during development did not include a JDK, so full compilation could not be executed there. Java syntax was checked using a parser, and the PlantUML diagrams were rendered successfully into SVG images.

5. Scenario Walkthrough

5.1 Customer Ordering Flow

1. The customer swipes an AP card and the kiosk identifies the user.
2. The kiosk displays available drinks and snacks from its synchronized menu cache.
3. The customer selects and customizes products.
4. The order service calculates the order total.
5. The payment service deducts the total from the AP card balance.
6. The transaction logger records the payment event.
7. The snack booth display receives the confirmed order for preparation.

5.2 Admin Update Flow

1. Snack booth admin staff update a product price or availability status.
2. The admin system saves the change in the central catalog.
3. The central catalog publishes a catalog event.
4. Each kiosk menu cache receives the event and updates its local product snapshot.

6. Compliance Assessment

Assignment Requirement	Status	Evidence in This Submission
High-level use case diagram with supporting descriptions	Satisfied	Figure 1 and section 2.2 provide the use case model and descriptions.
Class diagram representing the system design	Satisfied	Figure 2 provides the initial class diagram.
Three sequence diagrams for use cases	Satisfied	Figures 3, 4, and 5 cover browsing/customization, order payment, and admin update.
Updated class diagram with two to three design patterns	Satisfied	Figure 6 contains Singleton, Observer, and Factory Method.
Sequence diagrams showing pattern interaction	Satisfied	Figures 7, 8, and 9 show pattern interactions.
Implement patterns in Java or C++	Satisfied	Java skeleton exists under <code>src/main/java/omu</code> .
Evaluation of the solution and pattern choices	Satisfied	Sections 3 and 7 discuss pattern suitability, limitations, concurrency, and maintainability.

Critical appraisal report	Satisfied	Section 7 explicitly evaluates strengths, limitations, risks, and realistic future improvements.
Report structure and presentation	Mostly satisfied	The report is formatted for A4 printing and includes the group member names.

7. Critical Appraisal

7.1 Strengths of the Solution

The main strength of the solution is that it keeps the ordering workflow separated from product creation, catalog synchronization, payment, notification, and transaction logging. This makes the model easier to maintain because a failure in one responsibility can be isolated without reading the whole system. For example, payment issues are concentrated in `PaymentService` and `APCard`, while product synchronization is concentrated in `CentralCatalog` and the observer classes.

The design also matches the real scenario well. A central product catalog reflects the requirement that price and availability changes should be made centrally, while kiosk observers reflect the need for updates to affect all kiosk screens.

7.2 Suitability of the Design Patterns

The Singleton pattern is suitable for the central catalog because the project needs one authoritative menu source. The Observer pattern is suitable because kiosks should react to catalog updates automatically. The Factory Method pattern is suitable because drinks and snacks have different customization rules and future product families may be added.

The patterns are deliberately limited to the parts of the system where they add value. Payment, logging, and notification remain simple services because adding patterns there would increase complexity without improving the assignment solution.

7.3 Limitations and Risks

The implementation is intentionally skeletal. It demonstrates the object-oriented design and patterns, but it does not include a real database, a real AP card API, a kiosk user interface, or production-grade deployment. In a live campus environment, the system would need persistent storage, authenticated card access, retry logic for notification failures, and audit logs stored outside kiosk memory.

Concurrency is addressed at design level through a central catalog, thread-safe collections, locking, and synchronized card balance deduction. However, a production system would need database transactions or distributed consistency mechanisms to protect against simultaneous orders and catalog updates across different physical kiosk machines.

7.4 Balanced Evaluation

Overall, the solution is strong for an UML and pattern-focused mini project. It is maintainable, extensible, and directly connected to the assignment requirements. The biggest weakness is not the design structure, but the prototype depth: it proves the selected patterns rather than delivering a complete production kiosk application.

8. Conclusion and Future Enhancements

The proposed design satisfies the major functional requirements of the APU drinks and snack ordering kiosk scenario. It identifies the main actors and use cases, models the system through class and sequence diagrams, and applies suitable object-oriented design patterns to improve maintainability and extensibility.

The Singleton central catalog supports one authoritative product source, the Observer pattern supports kiosk synchronization, and Factory Method supports extensible product creation. Together, these patterns directly address the assignment's central concerns: multiple kiosks, central updates, and future menu expansion.

Future enhancements would include persistent database storage, a real AP card integration, a kiosk user interface, stronger error recovery for failed order notifications, automated unit tests, and distributed consistency controls for a real multi-kiosk deployment.

9. References

- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*.
- Oracle. (n.d.). *Java Platform Documentation*. <https://docs.oracle.com/en/java/>
- PlantUML. (n.d.). *PlantUML Language Reference Guide*. <https://plantuml.com/>
- Assignment brief: *OMU-Assignment-2601.pdf*.